部署实施说明

AI 心理健康助手前端 Docker Compose + Nginx + Cloudflare Tunnel

ai-vue-demo 生产化部署记录

项目目录：D:\G-NEW-next\ai-vue-demo

技术栈：Vue 3 + Vite 8 + Element Plus

部署环境：Windows 10 IoT Enterprise LTSC / Docker Desktop 28.5.1

实施日期：2026 年 11 月 9 日

目录

1. 部署目标与项目分析
2. 总体架构设计
3. 实施步骤
4. 遇到的六个问题与解决办法
5. 验证记录
6. 运维手册与命令速查
7. 后续建议（固定外网域名）

1. 部署目标与项目分析

1.1 目标

- 用 Docker Compose 把前端项目容器化，一条命令可构建、启动、重建。
- 用 Nginx 托管静态文件，并代理后端 API，使前端不依赖 Vite 开发服务器。
- 通过 Cloudflare Tunnel 暴露到公网，无需公网服务器和路由器端口映射。

1.2 项目技术特征

特征	说明	对部署的影响
Vue 3 + Vite 8 单页应用	源码经 `vite build` 产出 `dist/` 静态文件	需要多阶段构建，生产镜像不含 Node 运行时
vue-router history 模式	路由如 `/frontendknowledge`、`/auth/login` 由前端接管	Nginx 必须配置 SPA 回退，否则刷新页面 404
axios 请求前缀 `/api`	开发环境由 Vite proxy 转发到 `http://159.75.169.224:1235`	Nginx 需要等价的 `/api/` 反向代理
SSE 流式对话	`fetchEventSource('/api/psychological-chat/stream')`	Nginx 必须关闭代理缓冲，否则消息不能实时推送
图片地址使用配置常量	`baseURL + filePath`，原值为后端 HTTP 地址	外网 HTTPS 下会触发混合内容拦截，需要同源代理
文件上传（封面）	multipart/form-data 上传图片	Nginx 需要放宽 `client_max_body_size`

> **关键判断**：这是一个纯前端项目，没有需要容器化的数据库或后端服务。因此采用「多阶段构建 + Nginx 静态托管 + API 反向代理」的最小架构即可；后端 `159.75.169.224:1235` 保持原样，由 Nginx 转发。

2. 总体架构设计

2.1 运行时链路

```
公网用户
   │  HTTPS
   ▼
Cloudflare 边缘网络（trycloudflare.com）
   │  加密隧道（QUIC，出站连接，无需开公网端口）
   ▼
本机 cloudflared（Windows 进程）
   │  http://127.0.0.1:11000
   ▼
Docker 容器 ai-vue-demo（nginx:stable-alpine，容器内监听 80）
   ├── /          → /usr/share/nginx/html（Vue 构建产物，SPA 回退）
   ├── /api/      → http://159.75.169.224:1235（REST + SSE）
   └── /media/    → http://159.75.169.224:1235（图片资源，剥离 /media 前缀）
```

2.2 端口与访问地址

项目	值
宿主机映射端口	`11000` → 容器 `80`
局域网地址	`http://192.168.1.83:11000`
公网地址（临时）	`https://bureau-dimension-near-votes.trycloudflare.com`
后端 API	`http://159.75.169.224:1235`
容器名 / 镜像	`ai-vue-demo` / `ai-vue-demo:latest`
重启策略	`unless-stopped`（Docker 启动时自动拉起）

> **为什么不用 80 端口**：宿主机 80 端口已被既有容器 `spiritfruit-web` 占用。 首次启动时报错 `Bind for 0.0.0.0:80 failed: port is already allocated`， 因此改为 11000。后续如需占用 80，可停止该容器并把映射改为 `"80:80"`。

3. 实施步骤

3.1 编写 Dockerfile（多阶段构建）

构建阶段使用 Node 24 Alpine 安装依赖并打包；运行阶段只保留 Nginx 和静态文件，镜像更小、攻击面更低。

```
# ---------- 构建阶段 ----------
FROM node:24-alpine AS build

WORKDIR /app

COPY package.json package-lock.json ./
RUN npm ci

COPY index.html vite.config.js ./
COPY public ./public
COPY src ./src

RUN npm run build

# ---------- 运行阶段 ----------
FROM nginx:stable-alpine

COPY nginx.conf /etc/nginx/conf.d/default.conf
COPY --from=build /app/dist /usr/share/nginx/html

EXPOSE 80

CMD ["nginx", "-g", "daemon off;"]
```

设计理由

- **先拷贝 package 文件再 npm ci**：利用 Docker 层缓存，源码变动时不必重装依赖。
- **npm ci 而非 npm install**：严格按 lockfile 安装，构建可复现。
- **在镜像内构建**：不依赖宿主机已安装的 node_modules 或 dist，任何装有 Docker 的机器都能构建出一致产物。
- **Alpine 平台兼容性**：已确认 lockfile 中包含 `@rolldown/binding-linux-x64-musl` 与 `lightningcss-linux-x64-musl`，Vite 8 的 Rolldown 打包器可在 Alpine 正常使用。

3.2 编写 .dockerignore

避免把宿主机的 `node_modules`、`dist`、`.git` 和文档打进构建上下文，既加快传输又防止平台差异污染镜像。

```
node_modules
dist
.git
.gitignore
.vscode
*.md
Dockerfile
docker-compose.yml
.dockerignore
npm-debug.log*
```

3.3 编写 nginx.conf

这是部署的核心配置，覆盖 SPA 回退、API 代理、SSE 流式响应、图片同源代理、压缩与缓存。

```
# WebSocket / SSE 升级头
map $http_upgrade $connection_upgrade {
    default upgrade;
    ''      close;
}

server {
    listen 80;
    server_name _;

    root /usr/share/nginx/html;
    index index.html;

    # 上传文件大小限制（封面图上传等）
    client_max_body_size 50m;

    # 前端资源压缩
    gzip on;
    gzip_vary on;
    gzip_min_length 1k;
    gzip_comp_level 5;
    gzip_types text/plain text/css application/javascript application/json application/xml image/svg+xml;

    # ---------- 后端 API 反向代理 ----------
    # 与开发环境 vite proxy 一致：/api 原样转发到后端
    location /api/ {
        proxy_pass http://159.75.169.224:1235;
        proxy_http_version 1.1;

        # 等价于 vite 的 changeOrigin
        proxy_set_header Host $proxy_host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
        proxy_set_header Upgrade $http_upgrade;
        proxy_set_header Connection $connection_upgrade;

        # SSE 流式接口（AI 对话）必须关闭缓冲
        proxy_buffering off;
        proxy_cache off;
        proxy_read_timeout 3600s;
        proxy_send_timeout 3600s;
        chunked_transfer_encoding on;
    }

    # ---------- 图片等静态资源同源代理 ----------
    # 前端用 /media + 后端文件路径拼接，避免 HTTPS 下混合内容被拦截
    location /media/ {
        proxy_pass http://159.75.169.224:1235/;
        proxy_http_version 1.1;
        proxy_set_header Host $proxy_host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
    }
```

```
    # ---------- 带 hash 的静态资源长缓存 ----------
    location /assets/ {
        expires 30d;
        add_header Cache-Control "public, max-age=2592000, immutable";
        try_files $uri =404;
    }

    # ---------- SPA history 路由回退 ----------
    location / {
        try_files $uri $uri/ /index.html;
    }
}
```

关键配置说明

配置	作用
`try_files $uri $uri/ /index.html`	刷新 `/auth/login` 等前端路由时返回 index.html，由 vue-router 接管，避免 404。
`proxy_pass ...（无 URI 部分）`	保留原始 `/api/...` 路径转发，与 Vite 开发代理行为一致，后端无需改动。
`proxy_set_header Host $proxy_host`	等价于 Vite 的 `changeOrigin: true`，把 Host 改为目标后端地址。
`proxy_buffering off`	**SSE 必需**：关闭缓冲后，后端的流式 token 才能实时到达浏览器。
`proxy_read_timeout 3600s`	长连接对话可能持续很久，避免 Nginx 60 秒默认超时切断。
`location /media/` 且 `proxy_pass` 带尾斜杠	`/media/files/a.png` → 后端 `/files/a.png`，剥离前缀。
`client_max_body_size 50m`	允许封面图上传，Nginx 默认仅 1MB。
`/assets/` 长缓存	Vite 产物文件名带内容 hash，可安全设置 30 天 immutable 缓存。

3.4 编写 docker-compose.yml

```
services:
  web:
    build:
      context: .
      dockerfile: Dockerfile
    image: ai-vue-demo:latest
    container_name: ai-vue-demo
    ports:
      - "11000:80"
    restart: unless-stopped
    healthcheck:
      test: ["CMD", "wget", "-qO-", "http://127.0.0.1/"]
      interval: 30s
      timeout: 5s
      retries: 3
      start_period: 10s
```

健康检查使用 Alpine 自带的 BusyBox wget 请求容器内首页；容器状态最终为 `healthy`。

3.5 构建并启动

```
# 构建镜像并后台启动
docker compose up -d --build

# 查看状态
docker compose ps

# 查看日志
docker compose logs -f

# 停止并移除容器（镜像保留）
docker compose down
```

> **结果：**镜像构建成功，容器 `ai-vue-demo` 在 `0.0.0.0:11000` 启动并通过健康检查。

3.6 前端代码适配（3 处修改）

部署验证时发现三处代码只适用于开发环境或 HTTP 环境，必须修改才能在外网 HTTPS 下正常工作。

① 图片地址改为同源代理

文件：`src/config/index.js`

```
- export const baseURL = 'http://159.75.169.224:1235'
+ // 图片等静态资源统一走同源 /media 前缀，由 nginx 反向代理到后端，
+ // 避免 HTTPS 外网访问时的混合内容拦截
+ export const baseURL = '/media'
```

原因：公网入口是 HTTPS，而图片地址是 HTTP。浏览器会把页面中的 HTTP 图片视为混合内容并拦截或强制升级，导致封面图裂图。改为同源 `/media` 后，图片由同一 HTTPS 域名提供，再由 Nginx 转发给后端。

② 修复知识库封面写死的 localhost

文件：`src/views/User/frontendknowledge.vue`

```
- const getiamge = (url) => {
-   return url? `http://localhost:5173/`+url :" https://file.itndedu.com/psychology_ai.png"
- }
+ import { baseURL } from '../../config'
+ const getiamge = (url) => {
+   return url? `${baseURL}${url}` : "https://file.itndedu.com/psychology_ai.png"
+ }
```

原因：该地址指向 Vite 开发服务器，生产环境必然裂图；同时原字符串前多了一个空格，也会导致 URL 非法。

③ 调整 API 超时

文件：`src/utils/request.js`

```
- timeout: 1000,//请求的超时时间
+ timeout: 10000,//请求的超时时间（外网穿透有额外延迟，1s 过短）
```

原因：经公网隧道访问会增加往返延迟，1 秒超时会造成接口随机失败。

3.7 安装并启动 Cloudflare Tunnel

```
# 使用 scoop 安装（自动校验哈希）
scoop install cloudflared

# 启动快速隧道，指向本机 11000 端口
cloudflared tunnel --url http://127.0.0.1:11000 --no-autoupdate
```

cloudflared 通过出站 QUIC 连接与 Cloudflare 边缘建立隧道，本机无需公网 IP、无需路由器端口映射，也无需在防火墙开放入站端口。启动后会输出形如 `https://xxxx-xxxx-xxxx.trycloudflare.com` 的临时公网地址。

> **安装版本**：cloudflared 2026.10.7（scoop main bucket）。隧道注册成功日志：`Registered tunnel connection ... location=lax12 protocol=quic`。

4. 遇到的六个问题与解决办法

#	问题	现象 / 原因	解决办法
1	Docker 引擎未启动	`docker ps` 报错 `open //./pipe/dockerDesktopLinuxEngine: The system cannot find the file specified`，CLI 挂起等待	启动 Docker Desktop，并轮询 `docker info` 直到引擎返回版本 28.5.1 后再继续
2	80 端口已被占用	构建成功但启动失败：`Bind for 0.0.0.0:80 failed: port is already allocated`；占用者是既有容器 `spiritfruit-web`	宿主机端口改为 `11000:80`，不干扰现有服务
3	HTTPS 混合内容	公网为 HTTPS，页面图片地址为 `http://159.75.169.224:1235`，浏览器会拦截	新增 `/media/` 同源反向代理，`baseURL` 改为 `/media`，重新构建镜像
4	封面图写死开发地址	`http://localhost:5173/` 在生产环境不可达，知识库列表封面全部裂图	改用统一的 `baseURL` 常量拼接
5	API 超时过短	axios 超时 1000ms，经公网隧道偶发超时失败	调整为 10000ms
6	知识库接口 403	页面请求 `/api/knowledge/...` 返回 403，控制台报 AxiosError	对比直连后端同样返回 403，确认是后端要求登录态，**非部署问题**，登录后正常

> **排查方法示例：**遇到 API 异常时，分别请求「直连后端」和「经 Nginx 代理」两个地址，对比状态码与响应体。若二者一致，说明问题在后端或业务逻辑；若不一致，才需要检查代理配置。

5. 验证记录

5.1 接口与页面验证

验证项	地址	结果	说明
首页	`http://127.0.0.1:11000/`	**200**	返回 index.html
SPA 路由回退	`/auth/login`	**200**	刷新不 404
API 代理 POST	`/api/user/login`	**200**	返回 `{"code":"BUSINESS_ERROR","msg":"用户不存在"}`，与直连完全一致
图片同源代理	`/media/test.png`	**403**	请求已到达后端（后端鉴权返回 403），代理链路正常
局域网访问	`http://192.168.1.83:11000/`	**200**	同网段设备可访问
公网首页	`https://...trycloudflare.com/`	**200**	耗时约 2.3s
公网 SPA 路由	`/frontendknowledge`	**200**	正常返回页面
公网 API	`/api/user/login`	**200**	与直连后端响应一致
容器健康检查	`docker inspect`	**healthy**	健康检查通过

5.2 浏览器实测

- 通过公网 HTTPS 地址打开首页，页面完整渲染：导航栏、Logo、标语、按钮、页脚均正常。
- 访问知识库页面，静态资源（JS/CSS/图片）全部返回 200。
- 浏览器控制台除了后端鉴权导致的 403 外，没有其他 JavaScript 错误。
- 构建产物中已确认不再包含写死的 `159.75.169.224` 地址。

6. 运维手册与命令速查

6.1 Docker Compose

命令	用途
`docker compose up -d --build`	重新构建镜像并启动（代码改动后使用）
`docker compose up -d`	不重建镜像，直接启动
`docker compose ps`	查看容器状态与端口
`docker compose logs -f --tail=100`	实时查看 Nginx 访问/错误日志
`docker compose restart`	重启容器
`docker compose down`	停止并删除容器与网络（镜像保留）
`docker exec -it ai-vue-demo sh`	进入容器排查

6.2 Cloudflare 隧道

```
# 前台启动快速隧道（Ctrl+C 停止）
cloudflared tunnel --url http://127.0.0.1:11000

# 后台常驻（Windows 可用 start /b，或建议配置为计划任务/服务）
start /b cloudflared tunnel --url http://127.0.0.1:11000
```

临时性提醒：

- 快速隧道地址每次重启都会变化，且 Cloudflare 不保证可用性，仅适合演示与临时访问。
- 当前隧道进程属于本次操作会话，会话结束可能停止；长期运行请按下节配置固定隧道。
- 链接一旦公开，任何拿到地址的人都能访问，请勿在演示中放入敏感数据。

6.3 文件清单

文件	作用
`Dockerfile`	多阶段构建：Node 24 Alpine 打包 → Nginx 托管
`docker-compose.yml`	服务编排：端口映射、重启策略、健康检查
`nginx.conf`	SPA 回退、API/图片代理、SSE、压缩与缓存
`.dockerignore`	排除 node_modules、dist、.git 等构建上下文
`src/config/index.js`	图片基础路径改为 `/media`
`src/views/User/frontendknowledge.vue`	修复封面图写死 localhost 的问题
`src/utils/request.js`	API 超时 1s → 10s

7. 后续建议（固定外网域名）

若需要长期稳定地址，建议注册 Cloudflare 账号并创建具名隧道（Named Tunnel），绑定自己的域名。**核心步骤：**

1. 把域名 NS 托管到 Cloudflare。
2. 执行 `cloudflared tunnel login` 授权。
3. 执行 `cloudflared tunnel create ai-vue-demo` 创建隧道。
4. 配置 `config.yml`，把域名指向 `http://127.0.0.1:11000`。
5. 执行 `cloudflared tunnel route dns ai-vue-demo demo.example.com`。
6. 执行 `cloudflared service install` 注册为 Windows 服务，开机自启。

这样即可获得固定的 HTTPS 域名，且隧道进程常驻、无需手动启动。若偏好国内线路，也可改用 cpolar、natapp 或 frp（自有云服务器）方案，Nginx 侧配置无需改动。

> **总结：**本次部署以 Docker 多阶段构建产出可移植镜像，以 Nginx 统一处理静态托管、API/图片代理与 SPA 路由，以 Cloudflare Tunnel 解决无公网 IP 的外网访问问题。全部验证项通过，容器运行健康。

— 文档结束 —